

## Relatório 9 - Prática: Projeto Final (V)

Silas Ferré Sousa

### Introdução

O Airflow é uma ótima ferramenta para quando se necessita orquestrar pipelines em lote e tudo relacionado a dados. Apesar de existirem ferramentas de automação mais simples usando low-code atualmente como o N8N, o Airflow é indicado para cargas de pipelines grandes e complexas, que exigem granularidade no controle e maior customização. Além disso, o Airflow possui recursos bem interessantes como rastreamento com logs, agendamento, uma interface bem completa para acompanhar cada processo das automações, integração com diversas outras ferramentas ou sistemas, como buckets da AWS, Snowflake, banco de dados, entre outras.

### Arquitetura

De forma geral, no Airflow existem as DAGs (Grafo Acíclico Direcionado) que é um fluxo que define toda a sequência de execução da automação com início e fim. Cada DAG é composta por tarefas que possuem uma finalidade específica. A estrutura do Airflow composta neste trabalho foi construída via docker com containers e suas determinadas funções: um container chamado postgres é o container responsável pelo banco de dados de metadados. Outro container é o API Server que mostra toda a interface gráfica. Tem também o worker (executa as tarefas), triggerer (responsável por rodar operador assíncronos), Scheduler, redis e o Processor.

Para o projeto atual foi utilizado o ambiente WSL do Windows para configurar o docker e consequentemente o Airflow. Foi feita a instalação do docker-compose via terminal e para baixar o arquivo de configuração do Airflow foi utilizado o curl.

### Implementação

O projeto foi organizado em algumas pastas para melhor compreensão. As DAGs foram colocadas na raiz da pasta `./dags`. Dentro dela foi criada uma pasta chamada `include`, onde há o script [common.py](#) (responsável por guardar configurações em comum para as DAGs e onde também tem como guardar outras configurações caso deseje personalizar como a quantidade de retries, o owner, etc) e por fim os scripts [produtor.py](#) e [consumidor.py](#) (onde coloquei as funções que estão sendo usadas nas DAGs para não encher os scripts das DAGs de muitas funções e caso necessário serem reutilizadas no futuro).

Na implementação do projeto foram criadas duas DAGs, uma chamada `dag_requisicao_produtor.py` e outra chamada `dag_requisicao_consumidor.py`. Na primeira foram criadas duas tarefas, uma chamada `requisicao_fotos` que é um `PythonOperator` com uma função chamada `requisicao_foto` e outra tarefa chamada `disparar_consumidora` (que é um `TriggerDagRunOperator`). A função (localizada em `produtor.py`) da primeira tarefa é responsável por obter as imagens aleatoriamente via requisição da API do link da url (<https://random-d.uk/api/random>) fazendo o download de cinco imagens (foi escolhido essa quantidade para fins de demonstração e comprovação das requisições e do funcionamento do pipeline, podendo escolher mais) em um loop. As imagens são salvas em `/tmp/images` (volume compartilhado mapeado do contêiner do airflow para uma pasta do meu computador local, adicionado na parte de volumes do docker-compose.yaml da seguinte forma: `- ./minhas_imagens:/tmp/images`, conforme a Figura 1). Isso é necessário para o arquivo não existir somente no contêiner.

```

volumes:
  - ${AIRFLOW_PROJ_DIR:-.}/dags:/opt/airflow/dags
  - ${AIRFLOW_PROJ_DIR:-.}/logs:/opt/airflow/logs
  - ${AIRFLOW_PROJ_DIR:-.}/config:/opt/airflow/config
  - ${AIRFLOW_PROJ_DIR:-.}/plugins:/opt/airflow/plugins
  - ./minhas_imagens:/tmp/images

```

Figura 1 - Volume para salvar as imagens

Depois da requisição de pedir uma imagem aleatória para a API é feito uma requisição para obter o conteúdo binário da imagem (*requests.get(link\_foto, timeout=10)*), em seguida de forma resumida a imagem é adicionada e registrada no caminho especificado. Coloquei uma tratativa para que a tarefa seja concluída com sucesso se tiver a quantidade de imagens definidas (*QUANTIDADE\_IMAGENS = 5*, por exemplo).

Em seguida, nesta mesma função da primeira tarefa, é passado os caminhos das imagens via XCOM com o valor da chave (para ser identificada depois). A segunda tarefa é responsável por disparar a segunda DAG (a consumidora): é passado o nome da DAG que será disparada (*dag\_consumidora\_externa*), é definido o identificador *trigger\_run\_id* para a consumidora (para não ocorrer de ter valores de execuções diferentes), pois o XCom é associado a uma execução específica e pode acontecer de não achar o XCom. Já o *logical\_date* é necessário para agrupar a execução em um período lógico, pois o *ExternalTaskSensor* aguarda a mesma execução da produtora. Essa tarefa não aguarda a consumidora terminar, ou seja, só dispara e termina em seguida após disparar e será pulada caso a consumidora esteja executando com o mesmo *logical\_date*.

Na DAG da consumidora a primeira tarefa com *ExternalTaskSensor* aguarda a execução correspondente da DAG produtora atingir o estado de sucesso. Em *external\_dag\_id* é colocado o nome da DAG que será monitorada e que aciona ela. Outros comandos que coloquei, é a consumidora continuar quando a produtora estiver concluída com sucesso e falhar se a produtora falhar.

A outras tarefas são tarefas que simulam o treinamento. Elas são *PythonOperator*, pois coloquei para simular em funções separadas. Elas são colocadas em um script chamado [consumidor.py](#). A função da primeira tarefa chama *processar\_dados*, que é onde terá o *xcom\_pull* que obtém o XCom da DAG produtora (aqui preciso especificar o nome da tarefa que estou obtendo o valor e a chave também) que é uma lista dos caminhos das imagens. Em seguida, transforma cada item num objeto *path* para poder fazer algumas operações e lê elas. Depois é simulado o processamento das imagens. Na segunda tarefa a função simula o treinamento em épocas e na terceira tarefa simulação a validação.

No script para disparar a DAG de produtor, é necessário carregar o arquivo *.env* onde tem a senha e usuário do Airflow. Para executar o script instalei algumas dependências em um ambiente *.venv* (*requests* e *python-dotenv*) para poder executar diretamente via terminal. Coloco para rodar na raiz (no mesmo nível da pasta de *./dags*) com o seguinte comando: *python3 disparar\_produto.py*. O script solicita um novo token automaticamente a cada execução, e ele é usado para disparar a *dag\_produto\_externa*, em seguida confirma se teve o status de sucesso.

## Resultados

Na Figura 2, é mostrado o disparo da DAG de produtor por meio do script *disparar\_produto.py*. Mostra que está em fila devido não aguardar a DAG terminar, ou seja, só dispara.

```
(.venv) silas@silas:~/airflow-docker$ python3 disparar_produtores.py
DAG disparada com sucesso!
Estado: queued
```

Figura 2 - Comando para iniciar produtora

Como se pode observar a dag\_produtores\_externa é iniciada (Figura 3) e baixa as imagens (conforme os logs).

The screenshot shows the Airflow web interface for the 'requisicao\_fotos' task. The task is marked as 'Sucesso' (Success). The operator is 'PythonOperator'. The logs show the following messages:

```
2 [2026-09-05 21:33:25] INFO - DAG bundles loaded: dags-folder
3 [2026-09-05 21:33:25] INFO - Filling up the DagBag from /opt/airflow/dags/dag_requisicao_fotos
4 [2026-09-05 21:33:27] INFO - Imagem salva com sucesso em: /tmp/images/imagen_1.jpg
5 [2026-09-05 21:33:28] INFO - Imagem salva com sucesso em: /tmp/images/imagen_2.jpg
6 [2026-09-05 21:33:29] INFO - Imagem salva com sucesso em: /tmp/images/imagen_3.jpg
7 [2026-09-05 21:33:31] INFO - Imagem salva com sucesso em: /tmp/images/imagen_4.jpg
8 [2026-09-05 21:33:32] INFO - Imagem salva com sucesso em: /tmp/images/imagen_5.jpg
9 [2026-09-05 21:33:32] INFO - Done. Returned value was: None
```

Figura 3 - Primeira tarefa da dag\_produtores\_externa

Na Figura 4, mostra os logs da segunda tarefa e é interessante observar que ela finaliza em 21:33:33 e a dag\_consumidores\_externa é iniciada em 21:33:34 pela tarefa de esperar\_produtores (Figura 5). Na Figura 6 mostra o fluxo das duas DAGs.

The screenshot shows the Airflow web interface for the 'disparar\_consumidores' task. The task is marked as 'Sucesso' (Success). The operator is 'TriggerDagRunOperator'. The logs show the following messages:

```
2 [2026-09-05 21:33:33] INFO - DAG bundles loaded: dags-folder
3 [2026-09-05 21:33:33] INFO - Filling up the DagBag from /opt/airflow/dags/dag_requisicao_fotos
4 [2026-09-05 21:33:33] INFO - Triggering Dag Run. trigger_dag_id=dag_consumidores_externa
5 [2026-09-05 21:33:33] INFO - Dag Run triggered successfully. trigger_dag_id=dag_consumidores_externa
```

Figura 4 - Segunda tarefa da dag\_produtores\_externa

The screenshot shows the Airflow web interface for the 'esperar\_produtores' task. The task is marked as 'Sucesso' (Success). The operator is 'ExternalTaskSensor'. The logs show the following messages:

```
2 [2026-09-05 21:33:34] INFO - DAG bundles loaded: dags-folder
3 [2026-09-05 21:33:34] INFO - Filling up the DagBag from /opt/airflow/dags/dag_requisicao_fotos
4 [2026-09-05 21:33:34] INFO - Poking for DAG 'dag_produtores_externa' on 2026-09-06T00:00:00.000000Z
5 [2026-09-05 21:33:34] INFO - Success criteria met. Exiting.
```

Figura 5 - Primeira tarefa da dag\_consumidores\_externa

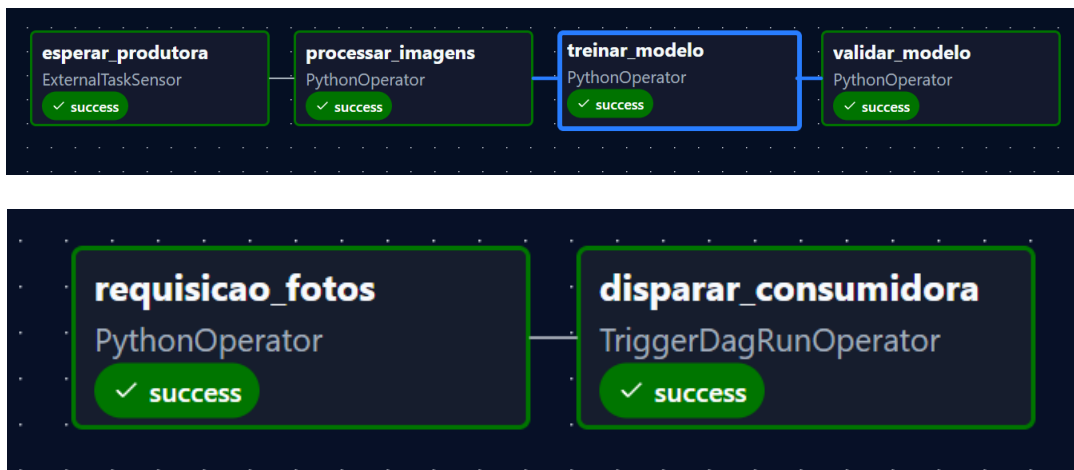


Figura 6 - Fluxo das DAGs

## Conclusão

Este projeto, assim como todo o curso foi bem interessante para aprender a função do Airflow, bem como as diferentes aplicações que pode-se fazer com ele. Com ele posso criar, organizar ou monitorar pipelines de dados com logs e diversas funcionalidades. Por exemplo, com o uso dessas duas DAGs (produtor e consumidor), pude aprender aplicação de operadores como TriggerDagRunOperator e ExternalTaskSensor que podem ser usados bastante em aplicações reais. Além disso, o uso de requisições via API Rest mostra que o Airflow pode ser iniciado por sistemas ou programas externos sem depender da própria interface ou de um agendamento feito na DAG. Apesar do treinamento ter sido simulado, conclui-se que a aplicação é viável usando o Airflow e o compartilhamento por meio do XCom dos caminhos mostra que o Airflow é flexível sem ter necessidade de armazenar as imagens no banco dele.

### Desafios encontrados e soluções adotadas:

- Compartilhamento das imagens entre containers;
- Correspondência e entendimento de run\_id e logical\_date;
- Autenticação na API REST.

### Melhorias futuras:

- Adicionar testes automatizados (unitários);
- Guardar as métricas em um bucket S3;
- Para projetos reais, utilizar uma quantidade maior de imagens rotuladas e separar os dados em treino, validação e teste e organizar as imagens em batches, em vez de apenas percorrer elas.
- Adicionar alertas por e-mail ou Slack, caso tenha alguma falha.

### Possíveis aplicações reais:

- Coleta periódica de imagens para treinamento de modelos;
- Treinamento e validação periódica de modelos;
- Atualização automática de conjunto de dados/banco de dados periodicamente;
- Solicitar um arquivo via chat (como Slack), rodar a automação e retornar ele para o usuário.